DiaRisk

Diabetes risk classification with a neural network

Technical project description and results

Abdelrahman Mostafa Kamal | Data Science and AI, Cairo University | October 2026

71.43%	0.8076	51.85%	82.00%
Test accuracy	ROC-AUC	Recall, diabetic class	Recall, healthy class

1. Overview

DiaRisk is a binary classifier that estimates whether a patient has diabetes from eight routine clinical measurements. It is built as a single scikit-learn **Pipeline** (missing-value handling, median imputation, standardization and a two-hidden-layer multilayer perceptron), so preprocessing and model are trained, evaluated and saved together.

On a held-out, stratified test set of 154 patients the model reaches **71.43% accuracy** and a **ROC-AUC of 0.8076**. The ranking quality is solid, but at the default 0.5 decision threshold the model identifies only **28 of 54** diabetic patients. The main takeaway is that the model has useful signal that the default threshold does not exploit.

2. Data

The dataset (`diabetes.csv`) has 768 records, eight numeric predictors and a binary `Outcome` label. It matches the Pima Indians Diabetes Database: adult women of Pima Indian heritage, which limits how far the results generalize.

Feature	Description	Zero treated as missing
Pregnancies	Number of pregnancies	No (zero is valid)
Glucose	Plasma glucose concentration	Yes
BloodPressure	Diastolic blood pressure	Yes
SkinThickness	Triceps skinfold thickness	Yes
Insulin	Serum insulin	Yes
BMI	Body mass index	Yes
DiabetesPedigreeFunction	Family-history score for diabetes	No
Age	Age in years	No

Subset	Samples	No diabetes	Diabetes
Full dataset	768	500 (65.1%)	268 (34.9%)
Training (80%)	614	400	214
Test (20%)	154	100	54

Disguised missing values. A value of 0 in Glucose, BloodPressure, SkinThickness, Insulin or BMI is physiologically impossible and encodes an unrecorded measurement. These zeros are converted to NaN and imputed instead of being fed to the network as real values.

3. Pipeline and model

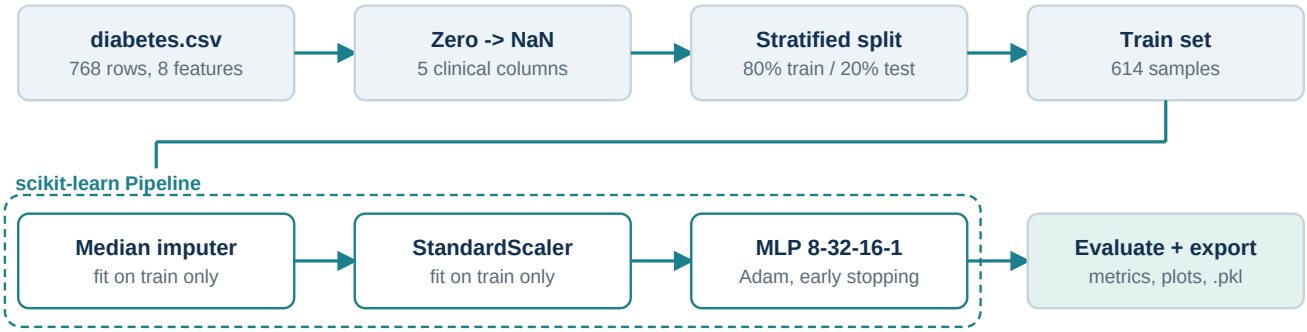


Figure 1. Data flow. Imputer and scaler statistics are learned from the training data only, which prevents test-set leakage.

Component	Setting
Architecture	8 inputs -> Dense 32 (ReLU) -> Dense 16 (ReLU) -> 1 output (logistic)
Trainable parameters	833 (288 + 528 + 17)
Optimizer / learning rate	Adam, initial learning rate 0.001
Loss	Binary cross-entropy with default L2 penalty
Early stopping	On; 15% of the training data held out for validation; patience 30 iterations
Max iterations	1000 (training stopped at 71)
Imputation / scaling	SimpleImputer(median) -> StandardScaler
Seeds	random_state = 42 for the split and the network

4. Training behaviour

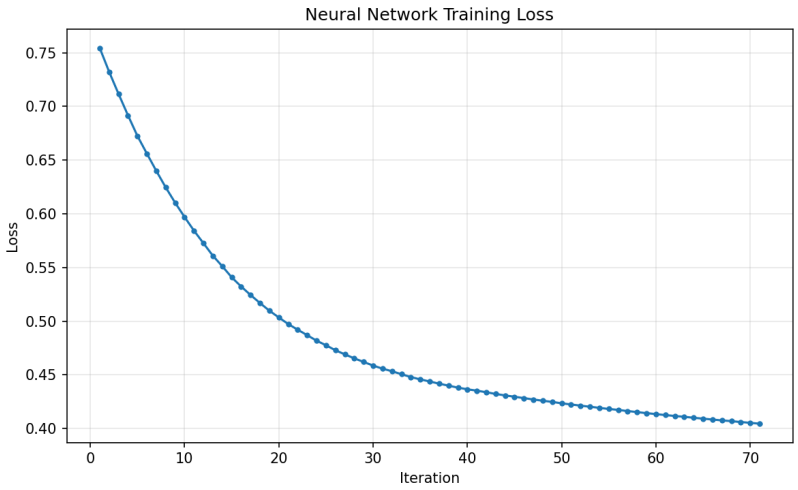


Figure 2. Training loss over 71 iterations (0.75 -> 0.4046).

The loss falls smoothly with no oscillation, steeply during the first 20 iterations and slowly afterwards, so the learning rate of 0.001 is stable. Training ended through **early stopping**: scikit-learn halts once the validation score has not improved for 30 consecutive iterations, which implies the validation score peaked around iteration 40. The training loss alone does not show this, so plotting the validation score is a worthwhile addition.

5. Test results

Held-out test set: 154 patients (100 without diabetes, 54 with diabetes).

Class	Precision	Recall	F1-score	Support
No Diabetes	0.7593	0.8200	0.7885	100

Class	Precision	Recall	F1-score	Support
Diabetes	0.6087	0.5185	0.5600	54
Macro average	0.6840	0.6693	0.6742	154
Weighted average	0.7065	0.7143	0.7084	154

	Predicted: No Diabetes	Predicted: Diabetes
Actual: No Diabetes	82 (true negatives)	18 (false positives)
Actual: Diabetes	26 (false negatives)	28 (true positives)

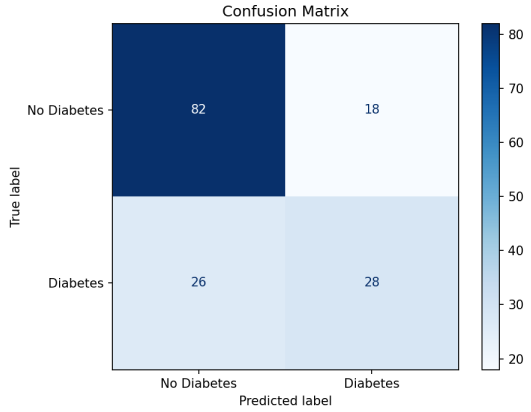


Figure 3. Confusion matrix.

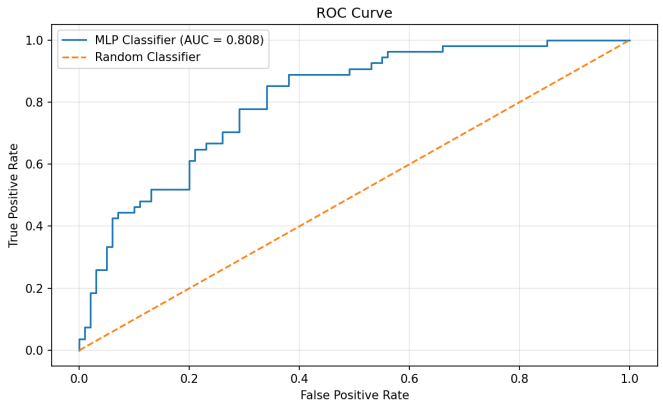


Figure 4. ROC curve (AUC = 0.808).

6. Analysis

Derived metric	Value	Meaning
Sensitivity (recall, diabetes)	51.85% (28/54)	Share of diabetic patients detected
Specificity	82.00% (82/100)	Share of healthy patients correctly cleared
Precision (PPV)	60.87% (28/46)	Share of positive predictions that are correct
Negative predictive value	75.93% (82/108)	Share of negative predictions that are correct
Balanced accuracy	66.93%	Mean of sensitivity and specificity
Majority-class baseline	64.94% (100/154)	Accuracy of always predicting 'No Diabetes'
95% CI for accuracy	approx. 64.3% to 78.6%	Normal approximation, n = 154

Key findings

- **Modest gain over the baseline.** 71.43% accuracy is only about 6.5 points above always predicting the majority class (64.94%). With a confidence interval of roughly +/-7 points, small differences from other models would not be distinguishable on this test set.
- **Good ranking, conservative decisions.** An AUC of 0.81 shows the scores separate the classes well, yet the default operating point (false-positive rate 0.18, true-positive rate 0.52) sits low on the ROC curve.
- **False negatives are the weak spot.** The model misses 26 diabetic patients, nearly as many as it finds (28). In a screening setting this is the costly error, because it delays diagnosis.
- **Threshold tuning should help.** Reading the ROC curve approximately, a false-positive rate near 0.29 corresponds to a true-positive rate near 0.78, and a rate near 0.34 to about 0.85, at the price of more false alarms.

7. Limitations

- Small dataset (768 rows) and a single 80/20 split; results will vary with another seed or split.
- No hyperparameter search, no cross-validation and no comparison with other models yet.
- Single population (adult women of Pima Indian heritage); performance elsewhere is unknown.
- Heavy missingness in some features (notably Insulin and SkinThickness) caps what any model can achieve.
- Default 0.5 threshold, unweighted loss and no probability calibration.
- Historical research data with no external validation. Not a medical device and not for diagnosis.

8. Next steps

Step	Goal
Stratified k-fold cross-validation	Stable estimates with confidence intervals for every metric
Baseline models	Compare against logistic regression, random forest and gradient boosting on identical folds
Threshold optimization	Raise diabetic-class recall by choosing the cutoff on validation data
Imbalance handling	Oversampling or similar, since MLPClassifier has no class weights
Hyperparameter search	Tune layer sizes, L2 strength and learning rate
Calibration and interpretability	Reliability curves, permutation importance or SHAP
Safer deployment	Move the zero-to-NaN step inside the pipeline and expose the model through a small API or demo

9. Reproducibility and outputs

Stack: Python, NumPy, pandas, Matplotlib, scikit-learn, joblib. **Seeds:** random_state = 42 for the split and the network.

File	Content
diabetes_mlp_classifier.pkl	Trained pipeline (imputer, scaler, MLP)
results.txt	Full metrics report
results_loss_curve.png	Training loss chart
results_roc_curve.png	ROC curve
results_confusion_matrix.png	Confusion matrix

Usage note. The zero-to-NaN conversion runs outside the saved pipeline. Before calling `predict_proba`, replace zeros with NaN in Glucose, BloodPressure, SkinThickness, Insulin and BMI. The pickle is tied to the scikit-learn version that created it and should only be loaded from trusted sources.